

RESEARCH ARTICLE



CrossMark

Danger-State Detection in Human-Robot Collaboration Using Deep Learning

Ameur Gargouri^{1,2}, Mohamed Karray^{*2} and Mohamed Ksantini¹

¹ATISP Lab, ENET'Com, University of Sfax, Sfax, Tunisia.

²ESME Research Lab, Ivry-sur-Seine, France.

*Corresponding author. E-mail: mohamed.karray@esme.fr.

Received xx xxx xxxx

Keywords: Human-robot collaboration, danger detection, temporal convolutional network, deep learning, wearable sensors, post-processing

Abstract

Collaborative robots operating in shared workspaces present inherent physical hazards, such as unexpected high-impact collisions, clamping and crushing forces. Deploying these robots in human proximity requires continuous monitoring via sensors to prevent operator injury and production interruptions. A core challenge in this context is the false-alarm trade-off: sensor outputs often lead to frequent emergency stops that erode productivity and operator trust. Current methods solve this problem either through the use of expensive force/torque sensors, visual processing pipelines, or shallow learning; nevertheless, no systematic analysis of current deep learning methods has been performed yet. To address these limitations, we propose, in this paper, a deep learning pipeline for binary danger-state classification (safe vs. danger) from high-frequency wearable Inertial Measurement Unit (IMU) data recorded during physical Human-Robot Collaboration (HRC). The proposed approach is based on three main components: (1) GPU-based feature expansion that computes velocity and acceleration from raw position data, creating 195 input channels without requiring extra force sensors; (2) a systematic comparison of five 1D deep learning architectures; and (3) a post-processing stage designed specifically to reduce false alarms. Evaluated with five-fold stratified cross-validation from 60 subjects, ConvNeXt 1D and Temporal Convolutional Network (TCN) demonstrate the best macro-F1 results (98.68% and 98.62% correspondingly), with the danger precision over 98% and danger recall over 97%. The post-processing stage alone reduces false positives by 87.5% at a cost of only 1.2 percentage points in recall, establishing a safer operating point for industrial deployment.

1. Introduction

HRC refers to shared-workspace scenarios in which a human operator and a robotic arm coordinate physical actions toward a common objective [15]. In these situations, it becomes imperative for the robot to keep on assessing whether the state of ongoing interaction is safe or whether there is a threat looming that calls for precautionary measures. Failing to detect a threatening situation could result in an injury, while false warnings could halt the process. The trade-off between sensitivity and specificity represents one of the key aspects of industrial safety under ISO/TS 15066 [8].

Existing collision detection techniques can be classified into three types. The hardware-based technique relies on force-torque or six degree of freedom force sensors to directly measure the contact forces [20]. The observer-based approach makes use of mathematical state estimators like the Extended

State Observer to estimate the forces acting on the robot using joint measurements without any extra sensors [18]. The data-driven approach trains the classifier on the sensor readings by performing signal transformations like wavelet decomposition [14] and multivariate fusion [5]. Recent trends include the use of digital twins to create collision situations in simulated environments [1] and the combination of zero-force control and deep learning [22].

In spite of these improvements, some issues still exist. Firstly, the introduction of the force torque sensor adds further cost and mechanical complexity. The observer design depends on an accurate Dynamic model, which is not guaranteed. Current machine learning-based methods mostly utilize shallow neural network or wavelet analysis on features. Additionally, the issue of false alarm due to inaccurate sensor readings has been poorly solved in existing works.

This paper addresses these gaps by proposing a deep learning pipeline that operates on wearable IMU data collected during physical HRC. Rather than relying on six-dimensional force sensors [20] or external cameras [7], the pipeline introduces a Graphics Processing Unit (GPU)-accelerated kinematic feature expansion that computes velocity and acceleration on the fly, augmenting the 65-channel IMU input to 195 spatio-temporal features and thereby encoding physical dynamics without additional hardware cost. Where recent collision detection studies have relied on wavelet networks [14], basic deep learning [22], or mathematical observers [18], this work systematically benchmarks five next-generation architectures originally designed for computer vision: Temporal Convolutional Network (TCN), ConvNeXt 1D, MLP-Mixer 1D, Transformer 1D and InceptionTime, adapted to one-dimensional time-series safety classification under identical training and evaluation conditions. To further improve prediction reliability, Test-Time Augmentation (TTA) is applied at inference time, a technique that remains virtually unexplored in the robotic collision literature despite its proven benefits in other domains [5]. Finally, temporal debouncing post-processing strategy is adopted, which uses three-tap median filtering with the best threshold to resolve the false alarms versus productivity dilemma. This way, the number of false alarms is decreased by 87.5%, while the accuracy of threat detection remains above 95%. The experiments are performed with strict five-fold stratified cross-validation on more than 69,000 windows gathered from 60 individuals, thus giving statistically robust evidence not achievable via digital twins [1] alone cannot guarantee.

The rest of this article is structured in the following manner. Section 2 considers prior works related to the problem of collision detection and safety monitoring in HRC. Section 3 covers the methodology of this study, including pre-processing, models architecture, training strategy, and post-processing. Section 4 shows experimental results and discussion. Section 5 concludes with implications and future directions.

2. Related Work

Safety monitoring in HRC has been addressed through sensor-based detection, model-based estimation and data-driven classification [10]. This section reviews recent contributions in each direction, highlighting the specific limitations that motivate the present work.

Force-torque sensing has been one of the basic methods of monitoring any physical collisions. Here, Wang et al. [20] presented a system of collision detection and force compensation using a six degrees-of-freedom force sensor attached to the end effector of the robot. By directly sensing the forces in all six degrees of freedom, accurate localization of contact can be achieved. However, the requirement of an expensive and difficult-to-calibrate force-torque transducer limits scalability to multi-robot systems, wherein each robotic arm will require its own array of sensors.

Expanding the detection of physical interactions to specific robot configurations, Gao et al. [6] explored collision detection in cable-driven parallel robots, an area where joint-based sensors cannot be used. Their approach utilizes cable tension variations to determine the interactions between the human user and the cable assembly. While successful in its own right due to the specific geometry of the cable

assembly robot, the algorithm is only suited to serial-link cobots and needs tension sensors which are not typically found in industrial cobots.

In order to reduce the need for external hardware, model-based disturbance estimation techniques have become popular. For example, Tonti et al. [18] proposed a quick collision detection algorithm that relies on an Extended State Observer (ESO) to estimate the disturbance forces based on joint positions and velocities without external hardware. The estimation of the disturbance force using ESO allows one to perform collision detection by applying the residual threshold technique. The main strength of this method is that no additional hardware needs to be installed, but the performance depends on how well the nominal dynamic model of the robot is defined.

Combining robot compliance control with learning-based collision detection provides another research avenue. Zhao et al. [22] use a combination of zero-force control and deep learning to detect collisions while maintaining the compliance of the robot. In this model, a neural network is trained to distinguish between intentional and accidental forces that lead to collision. Compliance control and collision detection can be combined to make an interesting contribution, but this method relies on joint torque information, which is not readily available for all inexpensive collaborative robots, and the deep learning approach used is fairly simple.

Considering a data-based approach, multimodal signal aggregation for increasing detection reliability has been studied. Fang et al. [5] introduce a multivariate collision detection algorithm using various sensors such as motor current measurements, joint angle sensors, and accelerometers used during HRC tasks. Such an approach utilizes information complementarity among different signals in order to achieve noise resistance of the system. However, the classification backend relies on conventional signal processing and shallow models and the study does not evaluate modern deep architectures that could learn richer cross-modal representations directly from the raw fused signals.

Focusing on time-frequency signal decomposition, Niu et al. [14] introduced Continuous Wavelet Networks (CWNet) that extract features from joint-level signals for efficient and transferable collision detection across different collaborative robots. The wavelet decomposition provides a principled multi-resolution representation of transient collision events and the authors demonstrate transfer learning between robot platforms. Nevertheless, the wavelet-based feature extraction constitutes a fixed preprocessing step that may discard discriminative patterns not captured by the chosen wavelet basis and the network architecture does not explore recent convolutional designs such as TCN or ConvNeXt.

Rather than reacting after physical contact occurs, vision-based proactive planning aims to prevent collisions entirely. Hoang et al. [7] addressed safety from a planning perspective by utilizing color and depth images to generate collision-free grasp trajectories before execution. Their vision-based pipeline evaluates candidate grasps against a reconstructed scene model, rejecting those that would result in contact with obstacles or the human operator. While proactive trajectory planning is complementary to reactive collision detection, this approach requires a calibrated RGB-D camera with a clear line of sight and cannot handle unforeseen events that occur during task execution.

Virtual workspace simulation has also emerged as a means to benchmark safety protocols safely offline. In this context, An et al. [1] developed a digital twin framework for offline collision analysis within HRC workspaces, enabling algorithm evaluation in a simulated environment before physical deployment. The digital twin copies both the kinematics of the robot and the workspace geometry in order to create synthetic collision situations that can be used to train and validate collision detection algorithms. Despite reducing the dangers involved with testing in the physical domain, simulation environments have been shown to lack the stochastic nature of human behavior, sensor noise, and other variables inherent in actual collisions.

In summary, prior work highlights the need for data-driven safety monitors that can achieve high precision without expensive hardware or complex dynamic models. A systematic comparison of modern deep architectures on high-frequency kinematic data, combined with temporal post-processing to mitigate false alarms, has not been reported. The present work directly addresses this gap.

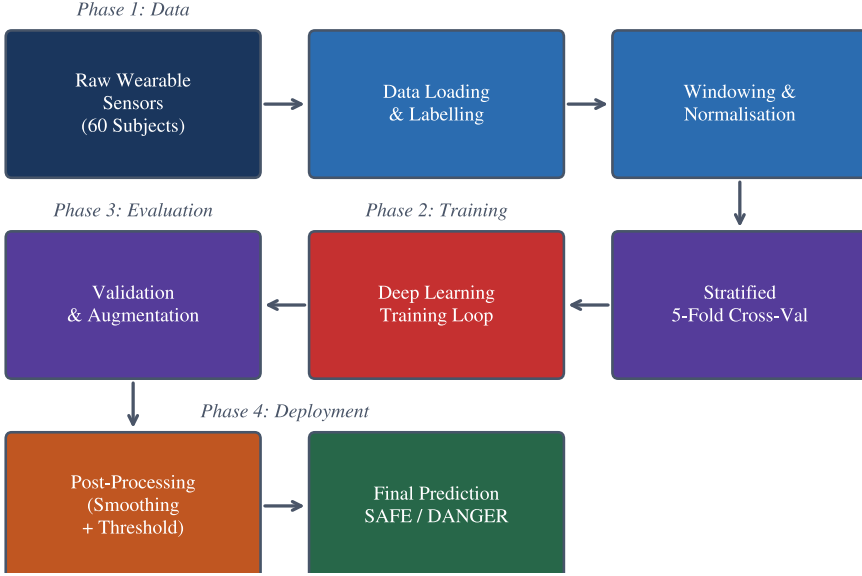


Figure 1. Proposed deep learning pipeline for danger-state detection in human-robot collaboration..

3. Methodology

The proposed solution follows the pipeline summarized in Figure 1. Raw IMU data is preprocessed, expanded via GPU-accelerated derivatives, passed through 1D deep learning classifiers and finally stabilized using a temporal debouncing post-processor.

3.1. Problem Formulation

Danger-state detection is formulated as a binary classification problem on sliding-window segments of multimodal IMU recordings. Let $\mathbf{X}_i \in \mathbb{R}^{C \times T}$ denote a window of T time steps across C sensor channels, with binary label $y_i \in \{0, 1\}$ indicating safe ($y_i=0$) or danger ($y_i=1$). A window is labeled as danger if any constituent time step carries a danger annotation, enforcing a conservative labeling policy:

$$y_i = \max_{t \in \{1, \dots, T\}} y_i^{(t)} \quad (1)$$

Given a model f_θ , the predicted class probability is $p_\theta(y=1 \mid \mathbf{X}_i)$. The predicted label is obtained by thresholding this probability at a tuned operating point τ :

$$\hat{y}_i = 1 [p_\theta(y=1 \mid \mathbf{X}_i) > \tau] \quad (2)$$

3.2. Data Collection and Dataset

All experiments in this work are evaluated on the DASIG (Dataset of Standard and Abrupt Industrial Gestures) benchmark [4], a standard corpus collected in a simulated industrial workstation to study human motion dynamics relevant to safety in human-robot collaboration scenarios. Data were acquired in the form of inertial signals collected at high frequencies from 60 healthy adults (36 males and 24

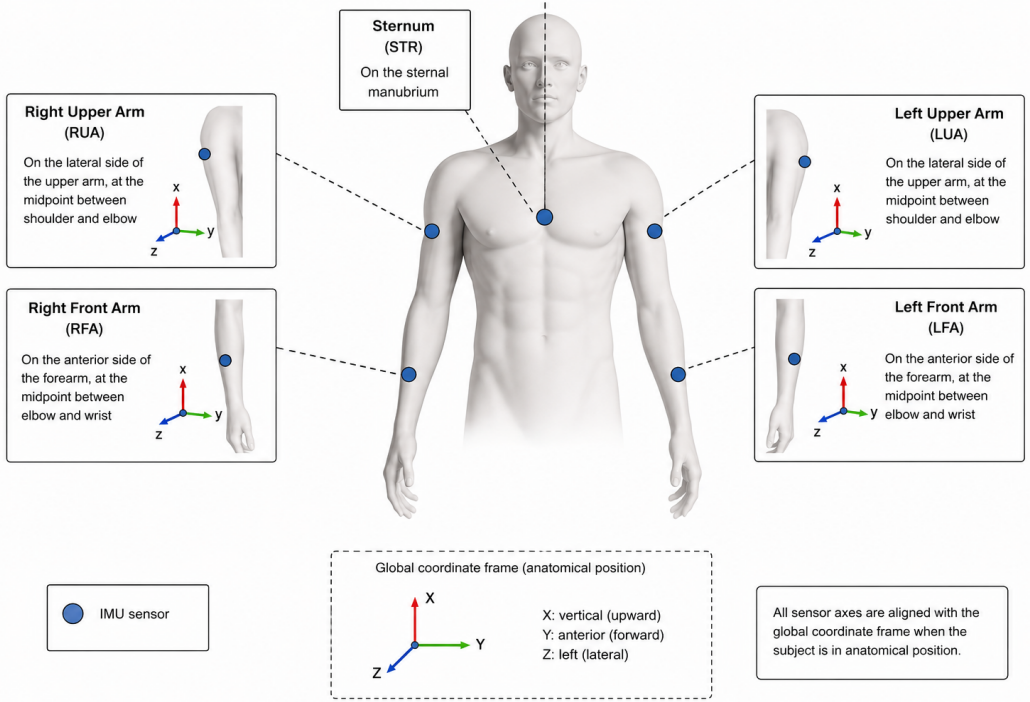


Figure 2. Anatomical placement of the five wireless MIMU sensors during the DASIG data collection experiments..

females aged between 20 and 59) carrying out industrial pick and place tasks. These subjects carried out three tests each to give complete interaction sequences.

Motion data were acquired at a sampling rate of 200 Hz using five wireless APDM Opal Magneto-Inertial Measurement Units (MIMUs) [2]. The five sensors were placed on key upper-body anatomical landmarks to capture upper-limb kinematics and torso motion: Sternum (STR), Right Upper Arm (RUA), Right Forearm (RFA), Left Upper Arm (LUA) and Left Forearm (LFA), as illustrated in Figure 2.

Each MIMU outputs 13 kinematic variables per time step: tri-axial linear acceleration ($\mathbf{a} \in \mathbb{R}^3$), tri-axial angular velocity ($\boldsymbol{\omega} \in \mathbb{R}^3$), tri-axial magnetic field intensity ($\mathbf{m} \in \mathbb{R}^3$) and a four-component orientation quaternion ($\mathbf{q} \in \mathbb{R}^4$). Across all five sensors, each window yields $5 \times 13 = 65$ synchronized input channels.

During each trial, subjects executed 30 standard pick-and-place gestures interleaved with four abrupt evasive movements (two triggered by visual alarms and two by acoustic alarms). Each experiment is accompanied by an Arduino event logging file that provides the time instances and numbers coding the stimuli involved: codes 2-5 correspond to the activation of green LEDs at stations SA-SD (standard gestures), codes 6-9 stand for the activation of red LEDs (visual alarm signals) and code 10 to the activation of acoustic buzzer.

The DASIG data set is made up of 180 individual files (60 subjects $\times$ 3 trials). Each file is mapped to a matrix of time series, which consists of 18,000 rows of time samples (at 200 Hz within ~ 90 s) and 66 columns. In these columns, there is one column of time stamps and 65 columns of sensor data (5 MIMUs $\times$ 13 variables). Event log files give multiclass labels to events (green light, red light, or buzzer) along with accurate time stamps and station identifiers. A binary target for classification is

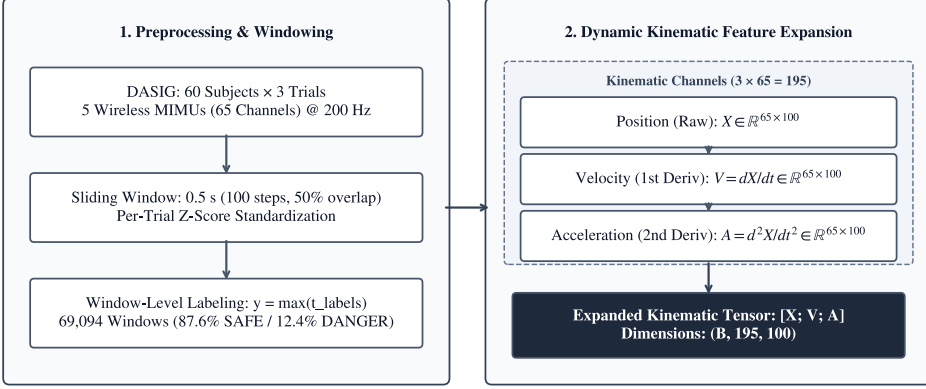


Figure 3. Data preprocessing and dynamic feature expansion on the GPU..

generated in this research where all the codes that trigger alarms (codes 6-10) are mapped to the *danger* category, while other normal gestures fall into the *safe* category.

3.3. Data Preprocessing and Dynamic Feature Expansion

Preprocessing transforms the raw DASIG recordings into a fixed-size tensor representation suitable for deep learning classifiers, proceeding through three sequential stages, as illustrated in Figure 3. First, raw recordings are segmented into windows of 0.5 s duration (100 time steps) with 50% overlap, producing 69,094 labeled windows. Second, each window is normalized per trial using z-score standardization:

$$x_{\text{norm}} = \frac{x - \mu_{\text{trial}}}{\sigma_{\text{trial}}} \quad (3)$$

where μ_{trial} and σ_{trial} are computed from the training partition of the corresponding trial. Third, window-level labels are assigned using the conservative maximum policy defined in Section 3: a window is labeled as danger if any constituent time step carries a danger annotation. The resulting dataset exhibits a class imbalance of 87.6% safe (60,559 windows) and 12.4% danger (8,535 windows).

Instead of relying entirely on positional data, the pipeline shown in Figure 3 performs preprocessing of each window by adding the first and second derivatives of the positional information in time via finite differences:

$$\mathbf{V}_i = \frac{d\mathbf{X}_i}{dt}, \quad \mathbf{A}_i = \frac{d^2\mathbf{X}_i}{dt^2} \quad (4)$$

The expanded representation $[\mathbf{X}_i; \mathbf{V}_i; \mathbf{A}_i] \in \mathbb{R}^{3C \times T}$ yields $3 \times 65 = 195$ channels which represent position, velocity, and acceleration. The augmentation process takes place on the GPU during training and testing. This removes the need for any kind of offline storage, as well as provides the network with kinematics information about human movements.

3.4. Algorithm Selection

Early iterations of this pipeline employed classical machine learning models, including Random Forest and XGBoost, trained on hand-crafted statistical features (mean, variance, skewness, kurtosis) extracted from each window. However, despite the good performance of these models in the original 65-channel

setting, there were two main problems associated with scaling these models to the new 195-channel kinematic settings.

The first problem is that traditional models work with fixed-sized vectors, which means that an extra step of feature extraction was required that removed the temporal dimension. The absence of the temporal dimension meant the absence of the sequence itself, which is important for safety-based transitions, since any increase in velocity can be indicative of an approaching collision. Second, the combinatorial growth of potential cross-channel interactions across 195 features makes manual feature design intractable and tree-based models cannot exploit the spatial locality of sensor groups (e.g., adjacent joint channels) without domain-specific feature grouping.

These observations motivated the transition to deep learning architectures that operate directly on the raw $(C \times T)$ tensor without manual feature extraction. The hierarchies learned by the deep models jointly represent temporal dynamics and cross-channel dependencies, leading to reduced information loss from the statistical abstraction.

As far as the area of deep learning is concerned, our approach involves the architectural decision in order to combine the three different kinds of inductive biases for a well-rounded comparison: (1) convolutional networks (TCN and ConvNeXt 1D) using local temporal dependencies through sliding windows and dilated receptive fields; (2) MLP-type networks (MLP-Mixer 1D) capturing global token and channel relationships in an unsupervised way without relying on convolution or attention; and (3) attention networks (Transformer 1D) utilizing self-attention to model long-range relationships. InceptionTime is considered as a multi-scale convolutional architecture often used for time series classification tasks. This combination includes the main architectural classes possible for sequential problems and allows us to draw conclusions on the inductive bias most suited for high-frequency kinematic safety signals.

Five architectures are adapted from their original two-dimensional formulations to operate on one-dimensional time-series input of shape $(B, 195, 100)$, where B is the batch size:

TCN [3]: A stack of six dilated causal convolutional blocks with kernel size 3 and exponentially increasing dilation factors ($d = 1, 2, 4, 8, 16, 32$), preceded by a 1×1 projection from 195 to 256 channels. Residual connections and batch normalization are applied in each block.

ConvNeXt 1D [12]: Four ConvNeXt layers which use depthwise separable convolution with kernel size 7 and inverted bottlenecks inspired by the two-dimensional network structure. Strided stem layer with kernel size 3 converts input channels from 195 to 128.

MLP-Mixer 1D [17]: Four mixer layers that use token-mixing and channel-mixing MLPs. The input is projected into 128-dimensional tokens via a patch embedding layer with kernel size 5 and stride 5.

Transformer 1D [19]: A four-layer encoder having four attention heads and a feed forward layer dimensionality of 256 along with learned positional encodings. The linear projection of the 195 input channels gives 128 dimensions.

InceptionTime [9]: Two inception modules with parallel convolutions at kernel sizes 1, 3 and 5 plus max-pooling, followed by adaptive average pooling.

All architectures terminate with adaptive average pooling, layer normalization and a linear classifier producing two-class logits.

3.5. Training Protocol

All models share an identical training protocol to ensure that performance differences reflect architectural properties rather than optimization variations. The training objective is focal loss [11] with focusing parameter $\gamma = 2.0$ and label smoothing $\epsilon = 0.01$:

$$\mathcal{L}_{\text{focal}} = -\frac{1}{N} \sum_{i=1}^N w_{y_i} (1 - p_{\theta}(y_i | \mathbf{X}_i))^{\gamma} \log p_{\theta}(y_i | \mathbf{X}_i) \quad (5)$$

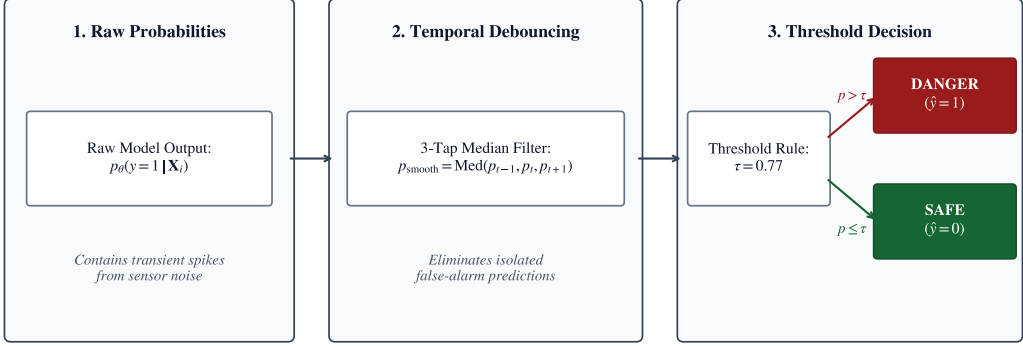


Figure 4. Temporal debouncing and threshold optimization on model probabilities..

where w_c is the class weight with respect to the inverse of frequency for the class c . Optimization was done using AdamW [13] with a learning rate of 10^{-3} , a weight decay of 10^{-4} and OneCycleLR scheduling [16]. Gradient clipping was done with a clipnorm value of 1.0. Batch size is 128 and training runs for 30 epochs. A weighted random sampler balances mini-batch class composition.

Data augmentation combines three strategies: (1) scale jitter that randomly scales each channel by a factor drawn from $\mathcal{U}(0.95, 1.05)$; (2) additive Gaussian noise with $\sigma = 0.01$; and (3) Mixup [21] with probability 0.3 and mixing coefficient $\alpha = 0.1$ drawn from a Beta distribution.

3.6. Evaluation Strategy

Model evaluation employs 5-Fold Stratified Cross Validation so that the class distribution remains same in all folds. Typically, each fold consists of 55,275 training instances and 13,819 validation instances. The checkpoint with the highest macro-F1 during the validation process is selected as the optimal model.

TTA is applied at inference time: a validation sample is run through the model four times (once normally and three times with Gaussian noise) to obtain the final score by averaging the softmax probabilities obtained from each pass.

Reported metrics include accuracy, macro-F1, danger recall (sensitivity) and danger precision. Confusion matrices are row-normalized to facilitate comparison across models.

3.7. Temporal Post-Processing

Raw model predictions are refined through two sequential post-processing steps designed to suppress transient false alarms without sacrificing detection latency.

First, a median filter of kernel size 3 is applied to the sequence of predicted danger probabilities. This temporal smoothing process filters out the single high-confidence predictions arising due to sensor noise and momentary ambiguous motion.

Second, the value of the threshold parameter τ is chosen to optimize this parameter within the range of $[0.50, 0.95]$, with an increment of 0.01, choosing the value of this parameter that ensures maximum danger precision with the constraint of 95% danger recall. With the choice of this threshold $\tau = 0.77$, the number of false positives is minimized considerably. The impact of this temporal post-processing is illustrated in Figure 4.

Table 1. Benchmark results with temporal post-processing ($\tau = 0.77$, median-filtered).

Rank	Model	Accuracy	Macro-F1	Danger Recall	Danger Precision
1	ConvNeXt 1D	99.43%	98.68%	97.13%	98.25%
2	TCN	99.41%	98.62%	97.14%	98.04%
3	MLP-Mixer 1D	99.28%	98.32%	95.88%	98.27%
4	InceptionTime	98.30%	96.22%	97.53%	89.62%
5	Transformer 1D	96.47%	92.50%	95.85%	79.71%

Table 2. Effect of temporal post-processing on TCN predictions.

Configuration	FP	FN	Precision	Recall
Raw output ($\tau = 0.50$)	1,324	142	83.9%	98.3%
+ Median filter + $\tau = 0.77$	166	244	98.0%	97.1%
Relative change	-87.5%	+71.8%	+14.1 pp	-1.2 pp

4. Results and Discussion

4.1. Comparison

Table 1 presents the benchmark results for all five architectures after temporal post-processing (median filter and $\tau = 0.77$). Models are ranked by macro-F1 score.

This demonstrates a clear order of performance in terms of CNN architectures (ConvNeXt 1D and TCN) performing best, followed by the mixed architecture (MLP-Mixer 1D), while the attention-only Transformer 1D obtains the poorest performance. Such an order is consistent with the well-known inductive bias of convolution operations in learning local temporal features from sequential data, which are common in kinematic danger signals.

ConvNeXt 1D produces the maximum macro-F1 accuracy of 98.68% with 148 false positives and 245 false negatives out of 69,094 windows (see Figure 5). The Temporal Convolutional Network (TCN) produces the almost equivalent macro-F1 accuracy of 98.62%, but it generates slightly higher numbers of false positives (166). Both models achieve more than 98% of macro-F1 accuracy; hence, only less than 2% of detected threats will be false positives.

The MLP-Mixer 1D achieves the precision of 98.27% and recall of 95.88%. It can be seen that the omission rate for true dangerous cases is about 4%. This result is due to the use of patch-wise tokenization in the mixer model, which might not be able to detect minute temporal variations between patches.

InceptionTime provides an excellent recall of 97.53%, yet it has significantly lower precision of 89.62%, which leads to 964 false positives. The multi-resolution parallel convolution mechanism used in the inception cell can successfully detect broader temporal trends, yet it produces more boundary uncertainties.

The Transformer 1D model, despite strong recall (95.85%), suffers from the lowest precision (79.71%) and produces 2,082 false positives. While the quadratic attention model is useful for capturing long dependencies, it does not appear to have the proper local inductive bias to handle the short dependencies found in safety-critical state changes using IMU data.

4.2. Impact of Temporal Post-Processing

Table 2 quantifies the effect of the median filter and threshold tuning on the TCN model, comparing raw model output (threshold 0.50) against the post-processed configuration.

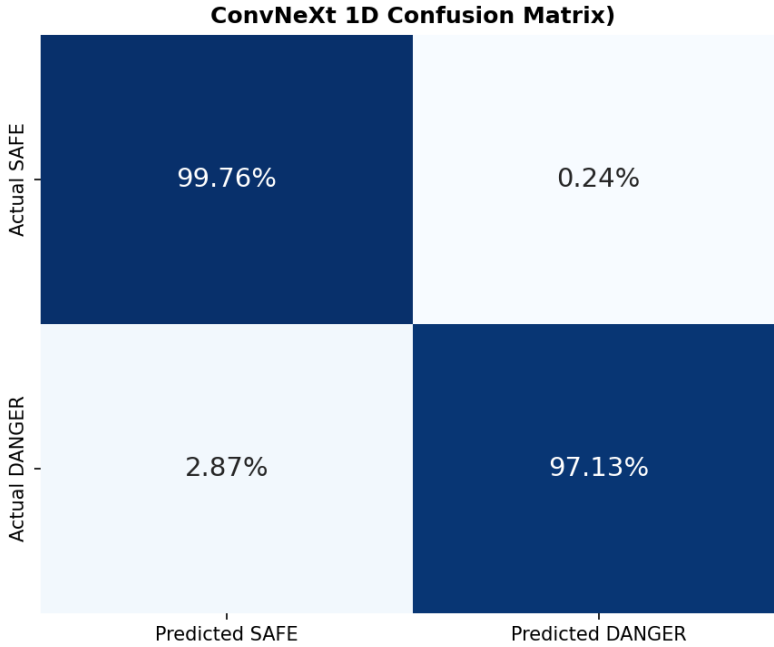


Figure 5. Row-normalized confusion matrix for the best-performing ConvNeXt 1D model..

Temporal post-processing leads to an 87.5% reduction in false positives (from 1,324 to 166), at the cost of a minor decline in recall of 1.2% (from 98.3% to 97.1%). Such a ratio is more beneficial for practical applications, because every false positive entails an unnecessary machine shutdown.

4.3. Discussion

Results of experiments reveal three important insights related to detecting danger states in HRC:

First, CNN architectures using dilated or depthwise convolutions have a high inductive bias when used to classify kinematic time-series data. Receptive fields of convolutional filters match well the temporal nature of the data about movements, which are defined by safety-critical transitions that happen in short consecutive periods of time. It is an opposite result to the modern tendency to develop models using attention and implies that the choice of architecture should depend on temporal properties of the target signal.

Second, dynamic feature expansion of 65 to 195 channels gives the network explicit velocity and acceleration information which would otherwise have to be extracted implicitly. Such solution helps to eliminate expensive force–torque sensors and reach accuracy rates higher than 98%, which matches those achieved with hardware-based solutions.

Third, temporal post-processing solves the problem of false alarms which, while acknowledged by many authors in HRC safety research, is rarely quantitatively analyzed. Median filtering reduces the impact of false detections due to noise, whereas the optimization of the threshold value shifts the trade-off point towards higher accuracy without loss of safety. This two-level solution does not depend on any particular classifier model.

5. Conclusion

Using collaborative robots in shared workspaces requires a careful balance between safety and productivity. Safety systems must detect genuine hazards to protect human operators, but frequent false alarms cause unnecessary emergency stops that slow down production and reduce operator trust.

To solve the problem stated above, this paper suggests a deep learning pipeline for online danger state detection utilizing the data from wearable sensors. The pipeline consists of three main parts: (1) feature expansion based on GPU-acceleration that allows obtaining the velocity and acceleration features from the position data and providing 195 input channels without requiring any force sensors; (2) comparison of five different one-dimensional deep learning models; and (3) post-processing aimed at reducing the number of false alarms.

The experimental evaluation with five-fold cross-validation on the DASIG dataset shows that convolutional models, namely, ConvNeXt 1D and TCN networks, give the best results with macro-F1 scores more than 98.6%. It is noteworthy that post-processing reduces the rate of false alarms by 87.5%, leading to danger precision higher than 98.0% and maintaining danger detection recall above 97.1%. Thus, it has been proven that the combination of wearable sensors and deep learning can ensure safe monitoring of human-robot cooperation without using expensive equipment.

In the future, the research plans further testing of the pipeline on unseen subjects, lightweight architectures analysis, and embedded implementation of the models to meet the requirements of real-time safety monitoring according to the ISO/TS 15066.

Author Contributions. All authors contributed to the study conception and design. Ameur Gargouri developed the methodology, implemented the software pipeline, performed data processing and experimental evaluations, and wrote the original draft of the manuscript. Mohamed Karray and Mohamed Ksantini supervised the research, guided the methodology, contributed to the analysis of results, and critically reviewed and revised the manuscript. All authors read and approved the final manuscript.

Financial Support. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflicts of Interest. The authors declare no conflicts of interest exist.

Ethical Approval. Not applicable.

Data Availability. The datasets analyzed during the current study are available from the corresponding author on reasonable request.

References

- [1] J. An, P. Bang, B. Jeon, *et al.* Development of digital twin for collision detection analysis within human-robot collaborative workspaces. *Journal of Computational Design and Engineering*, **11**, 1–15 (2024).
- [2] APDM Wearable Technologies 2021. Opal wireless inertial measurement unit specifications. APDM Inc., Portland, OR, USA.
- [3] S. Bai, J. Z. Kolter, and V. Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, **1803**, 01271 (2018).
- [4] E. Digo, M. Polito, E. Caselli, L. Gastaldi, and S. Pastorelli. A dataset of standard and abrupt industrial gestures recorded through mimus. *Robotics*, **14**(12), 176 (2025).
- [5] S. Fang, S. Liu, X. Wang, *et al.* A multivariate fusion collision detection method for dynamic operations of human-robot collaboration systems. *Journal of Manufacturing Systems*, **78**, 1–12 (2024).
- [6] H. Gao, C. Chevallereau, and S. Caro. Detection and management of human-cable collision in cable-driven parallel robots. *IEEE Robotics and Automation Letters*, **9**, 1–8 (2024).
- [7] D.-C. Hoang, A.-N. Nguyen, C.-M. Nguyen, *et al.* Collision-free grasp detection from color and depth images. *IEEE Transactions on Artificial Intelligence*, **5**, 1–14 (2024).
- [8] International Organization for Standardization 2016. Robots and robotic devices — collaborative robots. ISO/TS 15066:2016.
- [9] H. Ismail Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and F. Petitjean. InceptionTime: Finding alexnet for time series classification. *Data Mining and Knowledge Discovery*, **34**, 1936–1962 (2020).
- [10] A. W. Khan, M. Akhtar, S. M. Qureshi, M. Mustafa, N. A. Alsaleh, and I. Ahmad. A systematic review of safety-driven approaches in human–robot collaborative systems. *Sensors*, **26**(7), 2079 (2026).
- [11] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* 2017.
- [12] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 2022.
- [13] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *Proceedings of the International Conference on Learning Representations (ICLR)* 2019.
- [14] Z. Niu, T. Hassan, M. N. Boushaki, *et al.* Continuous wavelet network for efficient and transferable collision detection in collaborative robots. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, **54**, 1–13 (2024).
- [15] M. Selvaggio, M. Cognetti, S. Nikolaidis, S. Ivaldi, and B. Siciliano. Autonomy in physical human-robot interaction: A brief survey. *IEEE Robotics and Automation Letters*, **6**(4), 7989–7996 (2021).
- [16] L. N. Smith and N. Topin. Super-convergence: Very fast training of neural networks using large learning rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications* 2019.
- [17] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, *et al.* MLP-Mixer: An all-mlp architecture for vision. *Advances in Neural Information Processing Systems (NeurIPS)*, **34**, 24261–24272 (2021).
- [18] G. Tonti, S. Shakourzadeh, and C. Guarino Lo Bianco. A fast collision detection system based on an extended state observer. In *IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA)* 2024.
- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, **30**, 5998–6008 (2017).

- [20] Z. Wang, Y. Wang, Y. Feng, *et al.* Research on robot force compensation and collision detection based on six-dimensional force sensor. *SSRN Electronic Journal*, **1**, 1–12 (2025).
- [21] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. mixup: Beyond empirical risk minimization. In *Proceedings of the International Conference on Learning Representations (ICLR)* 2018.
- [22] B. Zhao, C. Wu, L.-j. Chang, *et al.* Research on zero-force control and collision detection of deep learning methods in collaborative robots. *Displays*, **88**, 102969 (2025).